Designed by Remus Teodorescu 2026

LEARNING OBJECTIVES

What You Will Be Able To Do

THE GOALS, IN PLAIN WORDS

- **write** — a PDE residual with autograd, and check it against an analytic derivative
- **assemble** — a composite loss, and say what each term is for
- **explain** — why the activation must be smooth, from the order of the residual
- **enforce** — a boundary condition softly and exactly, and state what each costs
- **choose** — between collocation, finite differences and the energy form
- **say** — why 105 sample points can constrain 2,751 parameters

WHAT TODAY ASSUMES

from L6.1	a loss term need contain no data
from Ex_03	a differential equation in a loss
from L4.1	why tanh, and not ReLU

YOU HAVE DONE THIS ONCE

Ex_03 put a damped oscillator in a loss and measured what it bought. The residual was given to you. Today you write it.

AND ON A PDE

Two independent variables instead of one, and boundary conditions instead of initial ones.

ROADMAP

The Logical Flow of This Lecture

1 Approximate

$u \approx \mathbb{N}\hat{w}$

A network replaces the unknown solution field

2 Differentiate

autograd

Exact derivatives of that network, to any order

3 Formulate

$\mathcal{F}(u) = 0$

PDE, boundary and initial operators

4 Penalise

$\mathcal{L} = \Sigma\, w_k \mathcal{L}_k$

Residuals assembled into one composite loss

5 Train

Adam → L-BFGS

Minimise $\mathcal{L}$ over the network parameters $\hat{w}$

- **five stages** — each a mathematical object, not an implementation detail
- the network is an ansatz, autograd an exact operator, the loss a functional
- **L7.2 reuses this pipeline** — only the operator and the conditions change

COURSE CONTEXT

Where L7 Sits: One Method, Five Applications

- **energy systems are governed by PDEs** — expensive to solve repeatedly
- and repetition is what design, control and optimisation require
- **every lecture that follows** — keeps this skeleton, changes only the physics

L8
Heat
Poisson and transient heat diffusion

L9
Fluids & batteries
Navier–Stokes; DFN electrochemical models

L10
Robotics
Vision-based, physics-informed but non-PDE

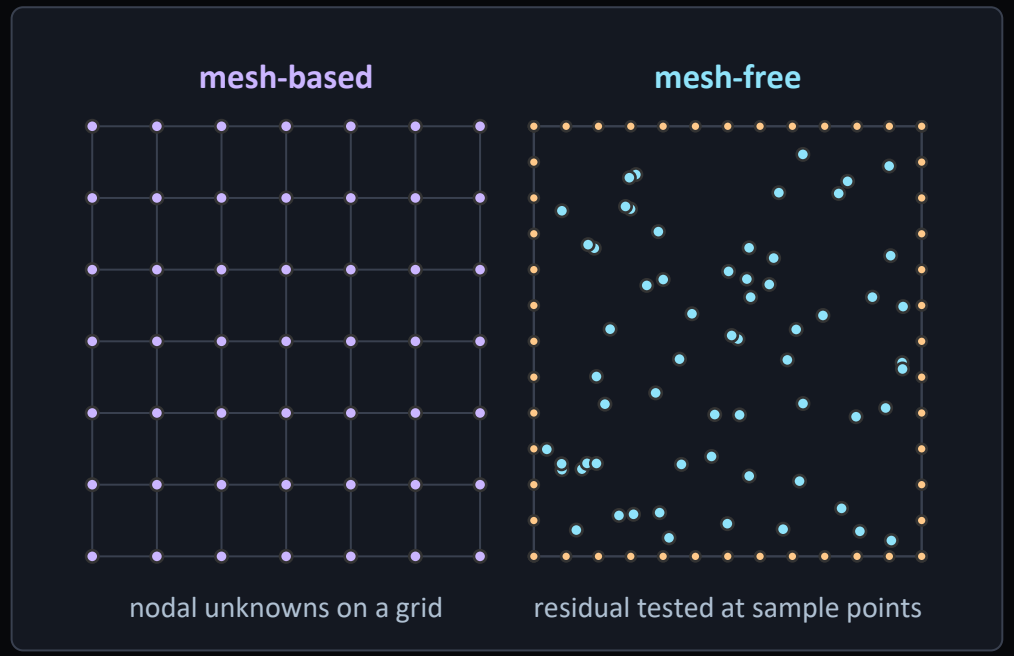
L11
Power grids
Optimal power flow acceleration

L12
Project
Implement the loss and the staged optimiser

The framing for this course: physics enters through the loss, not the architecture — which is why the same network, unchanged, solves heat conduction in L8 and a battery thermal model in a project.

Why Embed Physics in the Learning Problem?

- **classical solvers discretise first** — accuracy inherited from the mesh
- **their unknowns are nodal values** — cost grows exponentially with dimension
- a PINN makes the solution a differentiable function of the coordinates
- **so the PDE can be tested anywhere** — no connectivity required
- **and the physics is imposed, not learned** — which is why little data suffices



network + automatic differentiation + PDE residual = mesh-free solver

KEEP THE CLAIM HONEST

PINNs are not a drop-in replacement for FEM. On stiff, high-frequency or multi-scale problems classical solvers remain far more reliable — see the failure modes later in this lecture.

Raissi et al. (2019); Liu Ch. 1

Where PINNs Sit: The Modelling Spectrum

- PINNs are best understood not as a new kind of solver but as a bridge between two established traditions — and the loss function is the bridge.

Pure physics

FEM · FDM · spectral

Solve the governing equations
by discretisation.
No data required — but a mesh is.
Accuracy inherited from the mesh.

PINN

physics in the loss

Equations and data constrain
one continuous solution.
Mesh-free and differentiable.
Physics is a soft or hard constraint.

Pure data

ML surrogates

Fit observations with no
physical constraint.
Fast to evaluate — but free to
violate known laws.

no data, all structure

all data, no structure

This is why L7 spends its time on the loss function: the loss is where the physics actually lives.

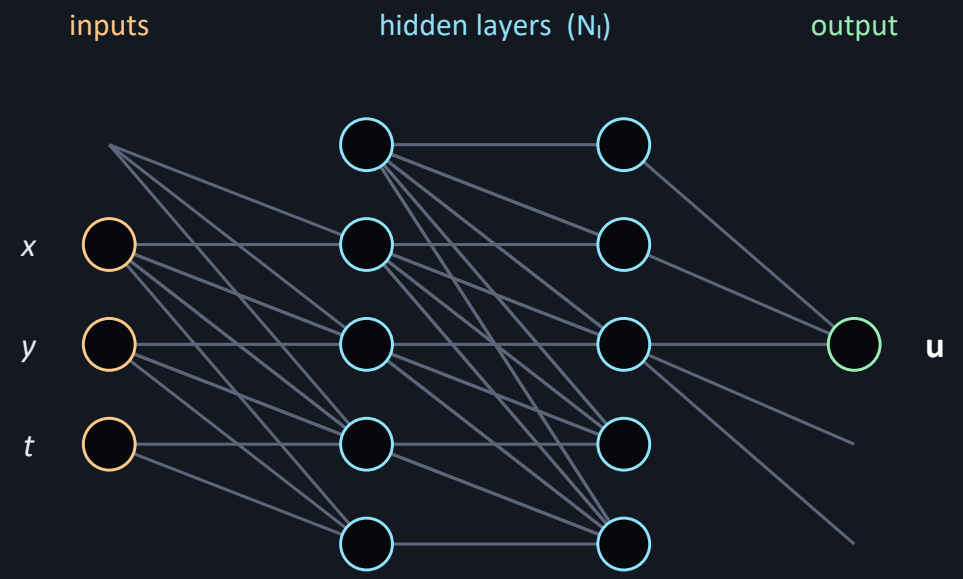
The Network as a Solution Ansatz

- **the unknown field becomes a parametric function** — a network with trainable parameters
- **inputs are the independent variables** — x, t, and any physical parameters
- each layer is an affine map then a smooth nonlinearity
- **differentiable to arbitrary order in its inputs** — which is the point
- **nothing is problem-specific yet** — the physics enters only through the loss

ANSATZ

$$u(\mathbf{x}, t, \mathbf{c}) = \mathcal{N}_{\hat{w}}(\mathbf{x}, t, \mathbf{c})$$

FEEDFORWARD ARCHITECTURE



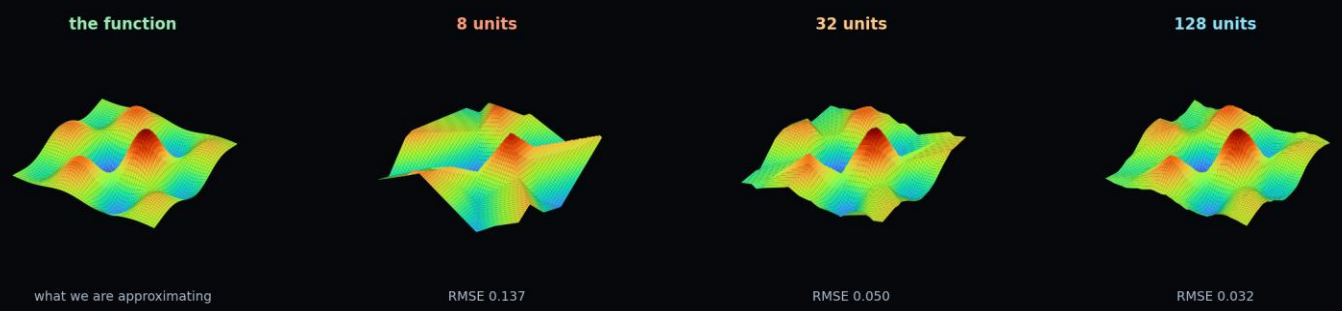
each edge carries a weight; each node a bias and activation σ

Universal Approximation Theorem

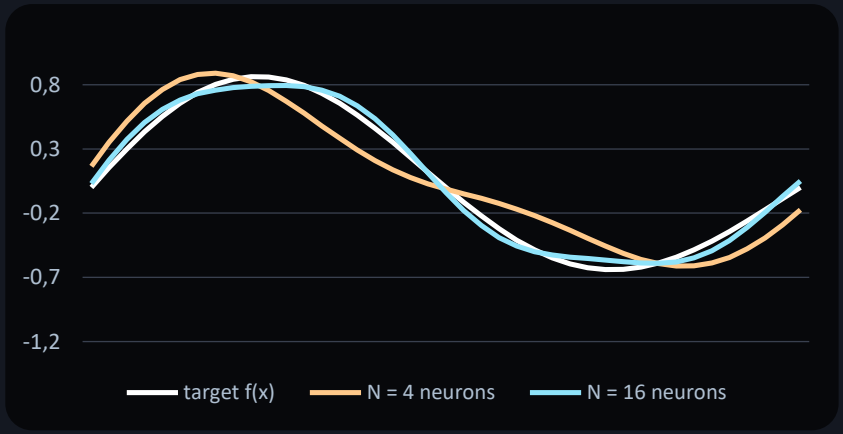
STATEMENT

$$\sup_{\mathbf{x}\in K} \left| u(\mathbf{x}) - \sum_{j=1}^{N} a_j \, \sigma\left(\mathbf{w}_j^{\mathsf{T}}\mathbf{x} + b_j\right) \right| < \varepsilon$$

- one hidden layer of finite width approximates any continuous function
- that is what licenses using a network as a solution surrogate at all
- **it is an existence result only** — silent on how many neurons, and on findability



WIDTH → ACCURACY



wider network → uniformly smaller error

Cybenko (1989); Hornik (1991); Lu et al. (2017)

Network Capacity and the Sampling Condition

PSEUDO-DIMENSION OF THE AFFINE SPACE

$$P^* = (p+1) + (N_n+1)\,N_L$$

SAMPLES–NEURONS CONDITION

$$N_{sample} \geq P^* \qquad N_{sample} \gg P^*$$

- **P* counts neurons, not weights** — the pseudo-dimension of the affine space
- fewer than P* independent sample points and the fit is under-determined
- **boundary and initial points count as extra** — the condition is on the interior set
- **note the asymmetry** — 105 sample points against 2,751 parameters
- it is the residual, not the data, that constrains the fit

WORKED EXAMPLE

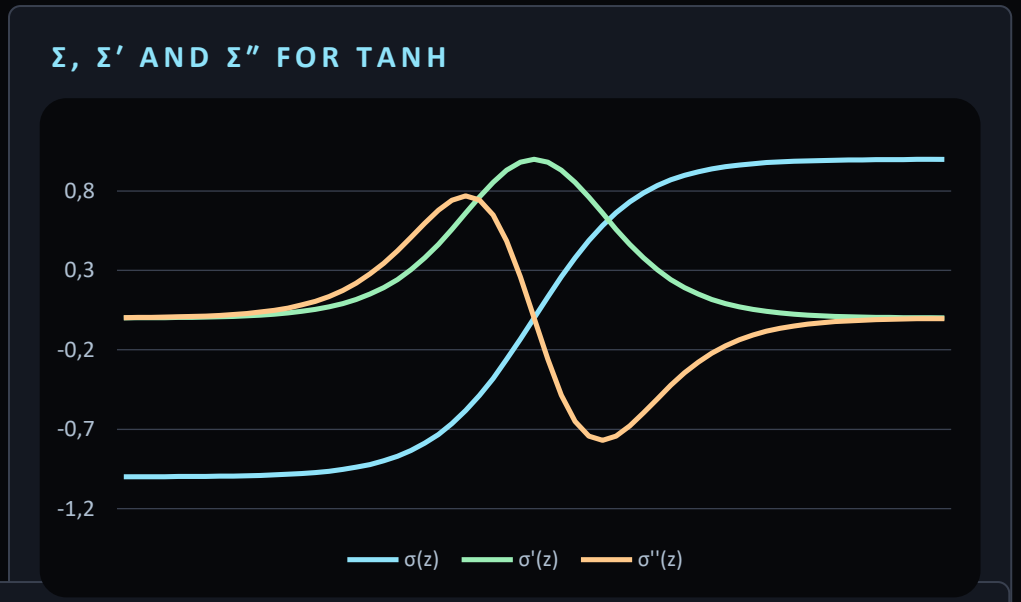
NETWORK	P*	PARAMS USED FOR	
50 × 2	**105**	2 751	Poisson, unit square
100 × 2	**206**	10 601	transient heat, wave
200 × 4	**808**	121 601	plate with hole, NBCs

P and the parameter count are different quantities — they differ here by factors of 26 to 150. The sampling rule uses P*.*

Liu Ch. 2.5.2, Eq. (2.19)

Why the Activation Must Be Smooth

- a residual of order k needs the network k-times differentiable
- **the activation is the only nonlinearity** — so it sets the ceiling
- **tanh is the default** — bounded, monotone, infinitely differentiable
- **ReLU is piecewise linear** — second derivative zero wherever defined
- smoothness is a requirement here, not a stylistic preference



ACTIVATION	σ(z)	σ″(z)	SECOND-ORDER PDE?
tanh	$\tanh(z)$	$-2\tanh z\,(1-\tanh^2 z)$	Yes — the default
sin / SIREN	$\sin(\omega_0 z)$	$-\omega_0^2 \sin(\omega_0 z)$	Yes — best for oscillatory
sigmoid	$1/(1+e^{-z})$	$\sigma(1-\sigma)(1-2\sigma)$	Yes — but saturates
ReLU	$\max(0, z)$	0 almost everywhere	No — residual collapses

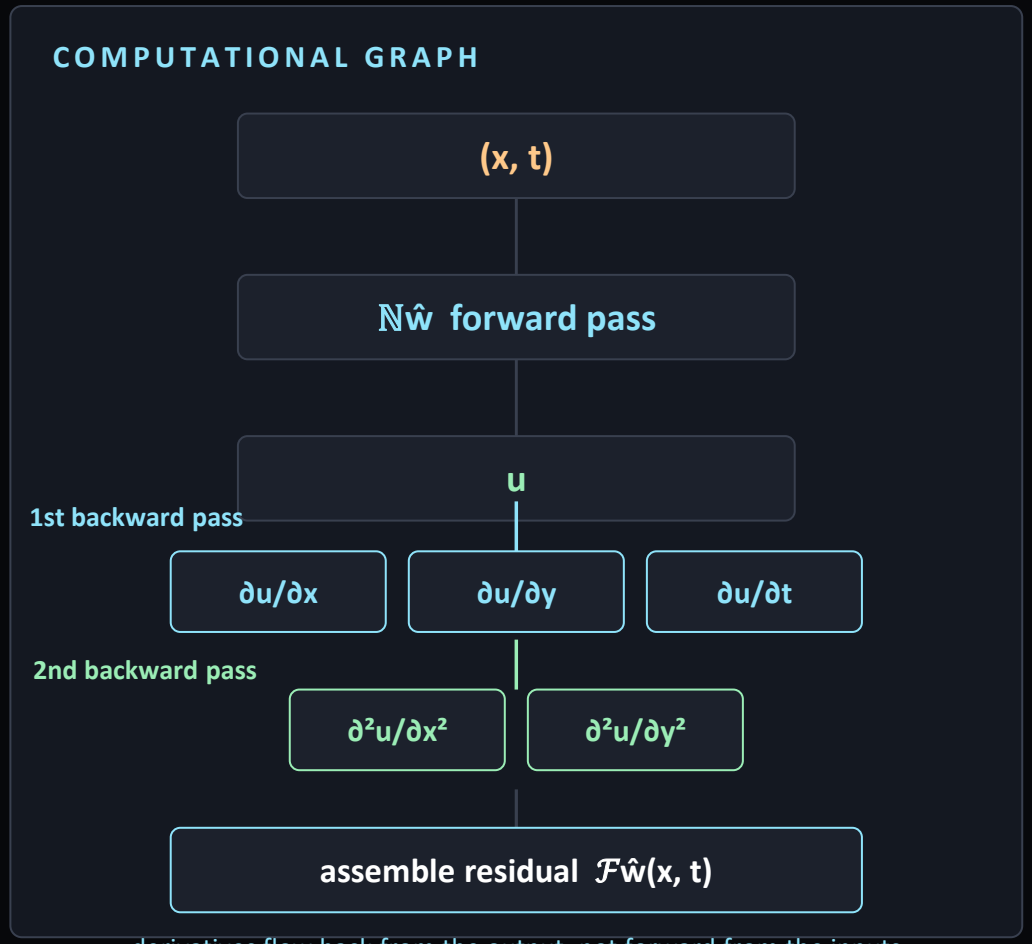
Liu Ch. 4.7; Prince UDL Ch. 3.1–3.2

Automatic Differentiation (Autograd)

- autograd applies the chain rule through the graph the forward pass built
- the derivatives are exact to machine precision
- no step size, no truncation error, no discretisation of the domain
- the same mechanism that gives the optimiser its gradients gives you du/dx
- **create_graph=True keeps the derivative differentiable** — second derivatives follow

RESIDUAL DERIVATIVES, IN THE BOOK'S STYLE

```
u = pinn(xyt)
grads = torch.autograd.grad(
    outputs=u, inputs=xyt,
    grad_outputs=torch.ones_like(u),
    create_graph=True)[0]
```



derivatives flow back from the output, not forward from the inputs

Autograd vs. Numerical and Symbolic Differentiation

SYMBOLIC

Manipulates expressions analytically.

Exact, but expression swell: derivative expressions grow combinatorially with order. Impractical for deep graphs.

NUMERICAL (FD)

Finite differences on a step h.

Truncation error $O(h^2)$ plus round-off error $O(\varepsilon/h)$ — the two conflict, so accuracy is bounded by an optimal h.

AUTOGRAD

Chain rule on the graph.

Exact to machine precision.
No step size to tune.
Cost scales with graph depth.
The PINN default.

FINITE-DIFFERENCE ERROR VS. STEP SIZE

- finite differences cannot be made arbitrarily accurate
- **shrink h and truncation error falls while round-off grows** — there is a floor
- autograd has no such floor
- which is why residuals are evaluated on the graph, not on a stencil

FORMULATION

The General PDE Operator Form

SYSTEM EQUATION

$$\mathcal{F}(u;\, \mathbf{x}, t, \mathbf{c}) = 0, \qquad \mathbf{x} \in \Omega,\ t \in [0,\, t_{end}]$$

- **F is a differential operator** — linear or nonlinear, acting on u
- u depends on x, t and the physical parameters c

ELLIPTIC
Poisson · L8
→ $\mathcal{F} = u_{xx} + u_{yy} + f$

PARABOLIC
transient heat · L8
→ $\mathcal{F} = u_t - \alpha(u_{xx} + u_{yy})$

HYPERBOLIC
wave · L7.2
→ $\mathcal{F} = u_{tt} - c^2(u_{xx} + u_{yy})$

one operator, three equations — only $\mathcal{F}$ and the admissible conditions change

DOMAIN AND ITS BOUNDARY

Liu Ch. 2.2, Eq. (2.1)

FORMULATION

Admissible Conditions: DBC, NBC, IC and IV

DIRICHLET (DBC)

$$u(\mathbf{x}, t) = \bar{u}(\mathbf{x}, t), \qquad \forall\, \mathbf{x} \in \partial\Omega_{DBC}$$

u prescribed on part of the boundary.

NEUMANN (NBC)

$$\mathcal{B}(\mathbf{x}, t, \mathbf{c}) = 0, \qquad \forall\, \mathbf{x} \in \partial\Omega_{NBC}$$

A derivative — flux, traction, strain — prescribed on the boundary.

INITIAL CONDITION (IC)

$$\mathcal{I}(\mathbf{x}, 0, \mathbf{c}) = 0, \qquad \forall\, \mathbf{x} \in \Omega$$

The state of the field at t = 0.

INITIAL VELOCITY (IV)

$$\mathcal{V}(\mathbf{x}, 0, \mathbf{c}) = 0, \qquad \forall\, \mathbf{x} \in \Omega$$

The time derivative at t = 0; required for second-order-in-time PDEs.

$\mathcal{F}$, $\mathcal{B}$, $\mathcal{I}$ and $^{\mathcal{V}}$ are all evaluated through the same network by autograd — the operators differ, the machinery does not.

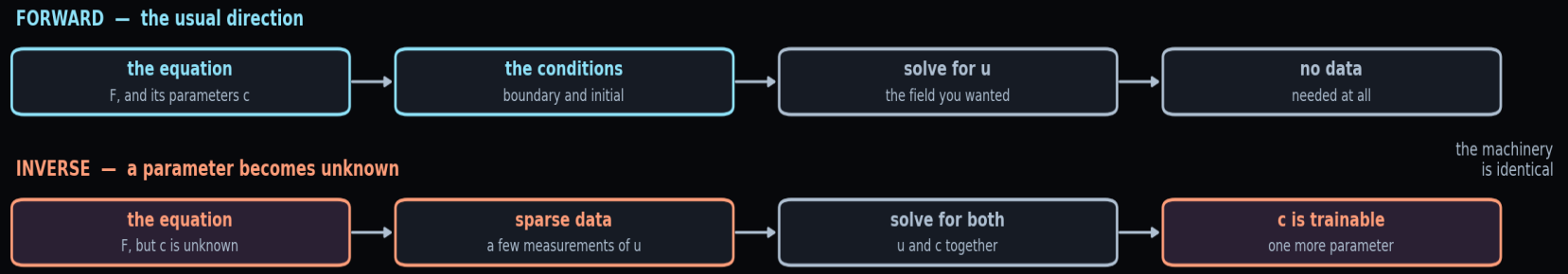
Forward and Inverse Problems

FORWARD PROBLEM

- ✓ operator $\mathcal{F}$ and parameters c
- ✓ boundary / initial conditions

INVERSE PROBLEM

- ✓ sparse measurements of u
- ✓ form of the operator $\mathcal{F}$



$\mathcal{L}^{\text{data}}$ is absent: training is driven purely by the residuals of $\mathcal{F}$ and the prescribed conditions. This is the setting for every example in L7.2.

Cost: one training run per parameter set.

Raissi recovers $\lambda_1 = 1$, $\lambda_2 = 0.01/\pi$ in Burgers'

WHY THIS IS WORTH HAVING — A BATTERY THERMAL EXAMPLE

A cell is instrumented with three surface thermocouples. The internal heat-generation rate and the thermal conductivity cannot be measured directly — but they are exactly the parameters *c* in the heat equation. Making them trainable recovers both from those three sensors, using the same network and the same loss. The classical alternative is to guess a value, run a forward solve, compare, and repeat.

The Composite Loss Functional

$$\mathcal{L} = \mathcal{L}_{data} + w_{PDE}\mathcal{L}_{PDE} + w_{DBC}\mathcal{L}_{DBC} + w_{NBC}\mathcal{L}_{NBC} + w_{IC}\mathcal{L}_{IC} + w_{IV}\mathcal{L}_{IV}$$

$\mathcal{L}^{data}$	$\mathcal{L}_{pde}$	$\mathcal{L}^{DBc}$	$\mathcal{L}^{Nbc}$	$\mathcal{L}^{Ic}$	$\mathcal{L}^{Iv}$
measurements	PDE residual	Dirichlet	Neumann	initial state	initial velocity
optional	*always present*	*if not hard-enforced*	*if flux prescribed*	*time-dependent*	*2nd order in t*

- every term is a mean-squared residual over its own sample set
- so all six are non-negative, and vanish together on the exact solution
- **the PDE term is always present** — which others appear is set by the problem

Liu Ch. 2.2.2, Eq. (2.6)

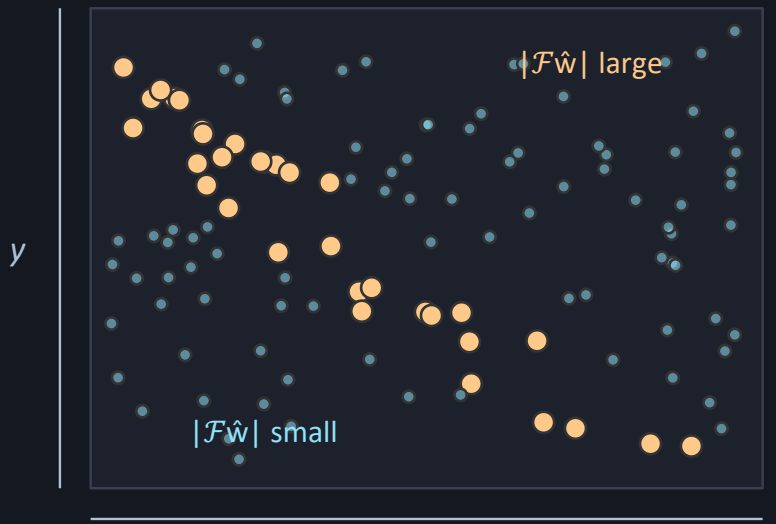
The PDE Residual Term

COLLOCATION FORM

$$\mathcal{L}_{PDE} = \frac{1}{n_{PDE}} \sum_{i=1}^{n_{PDE}} [\mathcal{F}_{\hat{w}}(\mathbf{x}_i, t_i)]^2$$

- the residual is what remains when the network is substituted into the equation
- **squared point by point, then averaged** — so the term is non-negative
- the collocation points live strictly inside the domain
- they carry no boundary information whatsoever

RESIDUAL MAGNITUDE OVER THE DOMAIN Ω



Reading this figure. The axes are the spatial domain Ω — this is a map of where the network sits, not a loss curve. Each dot is one collocation point. Marker size and colour encode $|\mathcal{F}\hat{w}|$, the absolute value of the residual at that point: how much the network violates the PDE there, in the units of the equation itself.

Dirichlet Conditions: Soft Enforcement

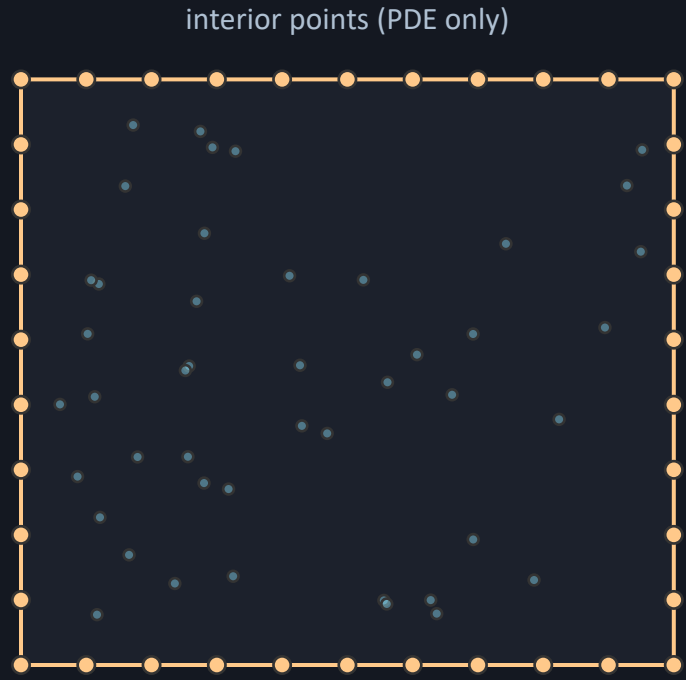
BOUNDARY RESIDUAL

$$\mathcal{L}_{DBC} = \frac{1}{n_{DBC}} \sum_{i=1}^{n_{DBC}} [u_{\hat{w}}(\mathbf{x}_i, t_i) - \bar{u}(\mathbf{x}_i, t_i)]^2$$

- the prescribed value is compared with the network output on the boundary
- soft enforcement means only approximately satisfied
- it competes with the PDE residual for the same parameters

PHYSICAL EXAMPLE — A LIQUID-COOLED POWER MODULE
The baseplate sits on a coolant channel held at 40 °C. That is a Dirichlet condition — the temperature on that face is prescribed, not computed.

BOUNDARY SAMPLING



Hard Enforcement: the Trial Solution

TRIAL SOLUTION ON A RECTANGLE [A,B] × [C,D]

$$u_{trial}(x,y) = x(1-x)\,y(1-y)\,\mathcal{N}_{\hat{w}}(x,y)$$

- the polynomial factor vanishes on all four edges
- so the trial solution satisfies the condition by construction
- **the boundary loss term disappears entirely** — nothing to balance
- training is markedly easier, and the search space is smaller
- **the cost is generality** — the factor must be built per geometry

The factor is chosen per geometry; Chapter 3 trains a small network to represent it on irregular domains.

Trial solutions used in the book

$f_{DBC} = x(1-x)\,y(1-y)$

zero on all four edges

$f_{DBC} = x$

zero on the left edge only

$f_{DBC} = \sin(\pi x)\sin(\pi y)$

trigonometric alternative

Hard vs. Soft Enforcement, Measured

BENCHMARK PROBLEM

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + 2\pi^2 \sin(\pi x)\sin(\pi y) = 0$$

on the unit square, u = 0 on all four edges

$$u(x, y) = \sin(\pi x)\,\sin(\pi y)$$

What this equation is. Poisson's equation on the unit square with a manufactured source term. The source $2\pi^2\sin(\pi x)\sin(\pi y)$ is chosen so the exact solution is known in closed form — which is the whole point: it lets every error figure below be computed against truth rather than estimated. Physically this is steady-state heat conduction in a square plate with a smoothly distributed internal heat source and all four edges held at zero.

tanh network, 40 × 2 | P = 85 | 320 collocation + 240 boundary points | Adam then L-BFGS | seed 88*

METRIC	SOFT BC	HARD BC
Relative L_2 error	2.12e-02	**1.29e-04**
Max absolute error	5.14e-02	**2.73e-04**
Max error on boundary	5.14e-02	**0.00e+00**
PDE residual MSE	8.47e-03	**4.55e-04**

Hard enforcement is ~165× more accurate in relative L_2, at essentially the same cost.

WHAT THE NUMBERS CONFIRM

The soft-BC model's largest error sits exactly on the boundary it was penalised to satisfy — the penalty bought an approximation, not a guarantee.

The hard-BC trial solution makes that error identically zero by construction — measured, not asserted.

Neumann Conditions and the Normal Derivative

NEUMANN RESIDUAL

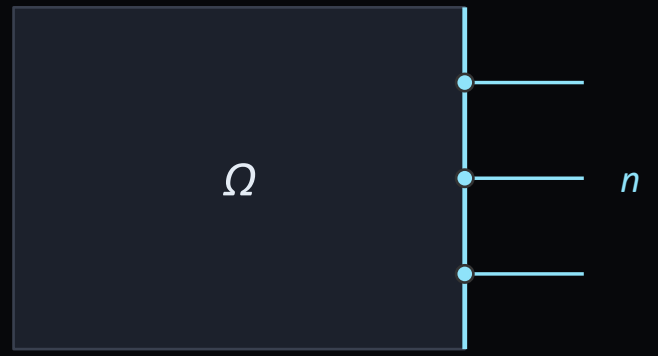
$$\mathcal{L}_{NBC} = \frac{1}{n_{NBC}} \sum_{i=1}^{n_{NBC}} [\mathcal{B}_{\hat{w}}(\mathbf{x}_i, t_i)]^2$$

- **B is a differential operator on the boundary** — it constrains a derivative, not a value
- flux, traction, insulation

NORMAL DERIVATIVE

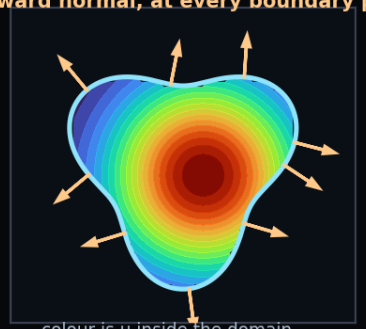
$$\frac{\partial u}{\partial n} = \nabla u \cdot \mathbf{n}$$

OUTWARD NORMAL ON ∂Ω



$\partial\Omega^{NBc}$

the outward normal, at every boundary point



colour is u inside the domain

$\nabla\hat{u}$	autograd gives the gradient
$\mathbf{n}$	the geometry gives the normal
$\nabla\hat{u} \cdot \mathbf{n}$	their dot product is the flux

$\mathcal{B}$ constrains that number, not $\hat{u}$ itself

Liu Ch. 2.2.2, Eq. (2.9)

Initial Conditions and Initial Velocity

INITIAL CONDITION

$$\mathcal{L}_{IC} = \frac{1}{n_{IC}} \sum_{i=1}^{n_{IC}} [\mathcal{I}_{\hat{w}}(\mathbf{x}_i, t_i)]^2$$

INITIAL VELOCITY

$$\mathcal{L}_{IV} = \frac{1}{n_{IV}} \sum_{i=1}^{n_{IV}} [\dot{\mathcal{I}}_{\hat{w}}(\mathbf{x}_i, t_i) - \dot{u}(\mathbf{x}_i, t_i)]^2$$

- both terms are sampled over the domain at t = 0
- **a first-order-in-time PDE** — the heat equation — needs only the initial condition
- **a second-order one** — the wave equation — is not determined by the initial state alone
- without the initial velocity the problem is under-determined
- **the dotted quantities come from autograd** — no separate discretisation

SPACE–TIME DOMAIN

IC + IV imposed on t = 0

Liu Ch. 2.2.2, Eq. (2.10)–(2.11)

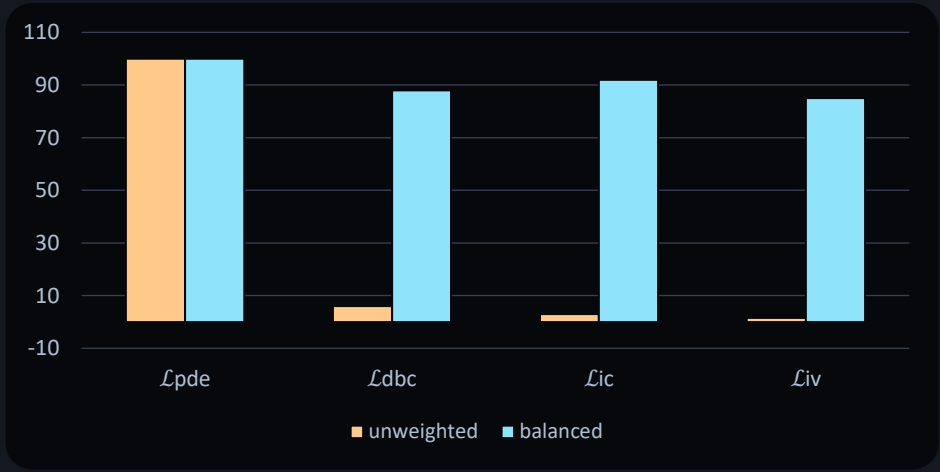
Weighting the Loss Terms

- the weights are hyperparameters, and the terms have different units
- so the raw magnitudes are not comparable
- when one term dominates, the optimiser ignores the others
- fixed weights are cheap, problem-specific and fragile
- **Liu's position** — restructure so the awkward term vanishes, rather than tune it

GRADIENT-NORM BALANCING

$$w_k \leftarrow \frac{\overline{\| \nabla_{\hat{w}} \mathcal{L}_{PDE} \|}}{\overline{\| \nabla_{\hat{w}} \mathcal{L}_k \|}}$$

GRADIENT MAGNITUDE PER TERM



Before balancing, the boundary and initial terms contribute almost nothing to the parameter update. After rescaling, every constraint influences training at a comparable rate.

Liu Ch. 2.2.3; Wang et al. (2021)

Three Ways to Evaluate the Residual

STRONG FORM

1

Collocation

Residual enforced pointwise, derivatives from autograd. Mesh-free; needs the highest differentiability of σ. The PINN default.

STRONG FORM, STENCIL

2

Finite difference

Do not differentiate the network at all. Instead evaluate it at neighbouring points a distance h apart and take differences — the classical FDM formula, applied to network outputs rather than to grid values.

WEAK FORM

3

Energy method

Minimise a potential-energy functional instead of a residual. Only first derivatives appear. Requires quadrature weights and essential BCs to be hard-enforced.

- all three minimise the same physics
- they differ in how much differentiation is pushed onto the network
- the weak form trades derivative order for integration

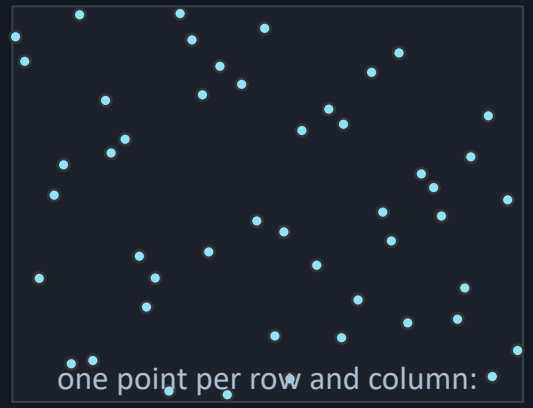
Collocation: Sampling the Strong Form

- the residual is enforced at a finite set of points
- **between them nothing is imposed** — so density limits accuracy

UNIFORM GRID

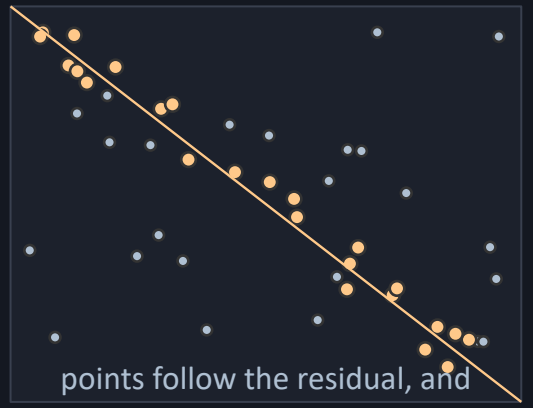
predictable, but density is uniform where it need not be

LATIN HYPERCUBE



one point per row and column: better coverage, fewer points

RESIDUAL-ADAPTIVE



points follow the residual, and concentrate at sharp features

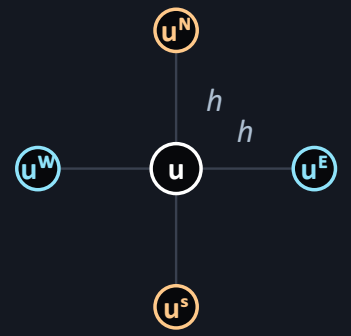
Finite-Difference Residuals (Central Scheme)

SECOND-ORDER CENTRAL DIFFERENCES

$$u_{,xx} \approx \frac{u(x-h,y) - 2u(x,y) + u(x+h,y)}{h^2}, \qquad u_{,yy} \approx \frac{u(x,y-h) - 2u(x,y) + u(x,y+h)}{h^2}$$

- **a trained network can be queried anywhere** — so classical stencils apply directly
- the Laplacian is assembled from four neighbouring evaluations
- the five network evaluations are each differentiable in the parameters
- **so training still works** — the graph just stays shallow
- **the trade is explicit** — truncation error returns, and h must be chosen

FIVE-POINT STENCIL



four extra network evaluations per collocation point

The Energy Method (Weak Form)

TOTAL POTENTIAL ENERGY FOR POISSON'S EQUATION

$$U_T(u) = \frac{1}{2}\int_\Omega \nabla u \cdot \nabla u \, dx - \int_\Omega u\, f \, dx - \int_{\partial\Omega_{NBC}} u \; t \, ds$$

STATIONARITY

$$\delta[U_T(u)] = \delta\left[U - W_f - W_t\right] = 0$$

Strong form
$-\Delta u = f$

—

Test function
multiply by v, integrate

—

Integrate by parts
order reduced to first

—

Energy functional
$U^T = U - Wf - W_t$

—

Minimise
$\delta U^T = 0$

- integration by parts moves one derivative onto the test function
- **so only first derivatives of u remain** — a weaker demand on the activation
- **the Dirichlet condition becomes essential** — it must hold for the principle to apply
- Neumann conditions appear as a boundary work term and need no loss

Liu Ch. 2.8, Eq. (2.45); Ch. 2.10

T3 Triangulation and Integration Weights

ELEMENT AREA

$$A = \frac{1}{2}\,|\mathbf{u}\times\mathbf{v}| = \frac{1}{2}\,|u_x v_y - u_y v_x|$$

NODAL WEIGHT

$$w_i = \sum_{T \ni i} \frac{A_T}{3}$$

QUADRATURE

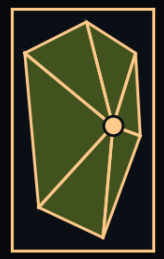
$$\int_\Omega g\; d\Omega \approx \sum_{i=1}^{n} w_i\; g(\mathbf{x}_i), \qquad \sum_{i=1}^{n} w_i = A_\Omega$$

- the domain is triangulated into three-node cells
- **the mesh does not store unknowns** — the network is still the solution

three-node cells over the domain

colour is the node's integration weight

one node, and the cells around it

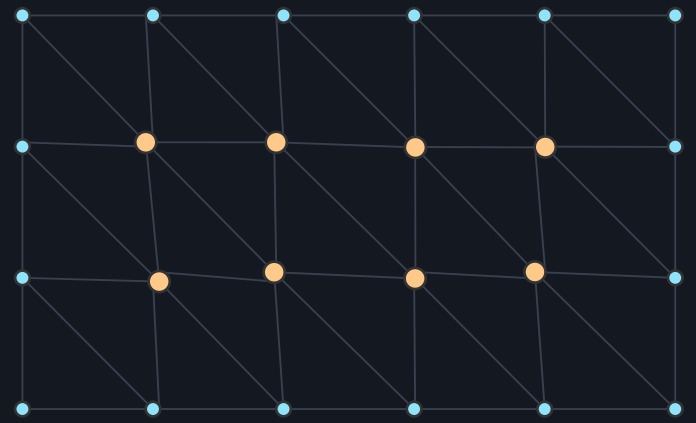


each gives it a third of its area

the weights sum to the domain area, so the quadrature is consistent on any mesh

94 triangles, 50 nodes

T3 MESH WITH NODAL WEIGHTS



marker size ∝ nodal weight w_i

TRAINING

The Training Loop

1 Sample

Draw interior, boundary and initial points; attach weights if the residual is integral.

2 Forward

Evaluate $\mathbb{N}\hat{\mathbf{w}}$ at every sample point in one batched pass.

3 Differentiate

Autograd produces all spatial and temporal derivatives required by $\mathcal{F}$, $\mathcal{B}$, $\mathcal{I}$ and $^{\mathsf{v}}$.

4 Assemble

Form each squared residual and combine into the composite loss $\mathcal{L}$.

5 Update

Backpropagate $\partial\mathcal{L}/\partial\hat{\mathbf{w}}$ and step the optimiser: Adam first, then L-BFGS.

6 Adapt

Optionally resample where the residual is largest, then repeat until the criterion is met.

Only step 5 touches the parameters. Everything else is the construction of a scalar functional — which is why the same loop trains every PDE in L7.2 unchanged.

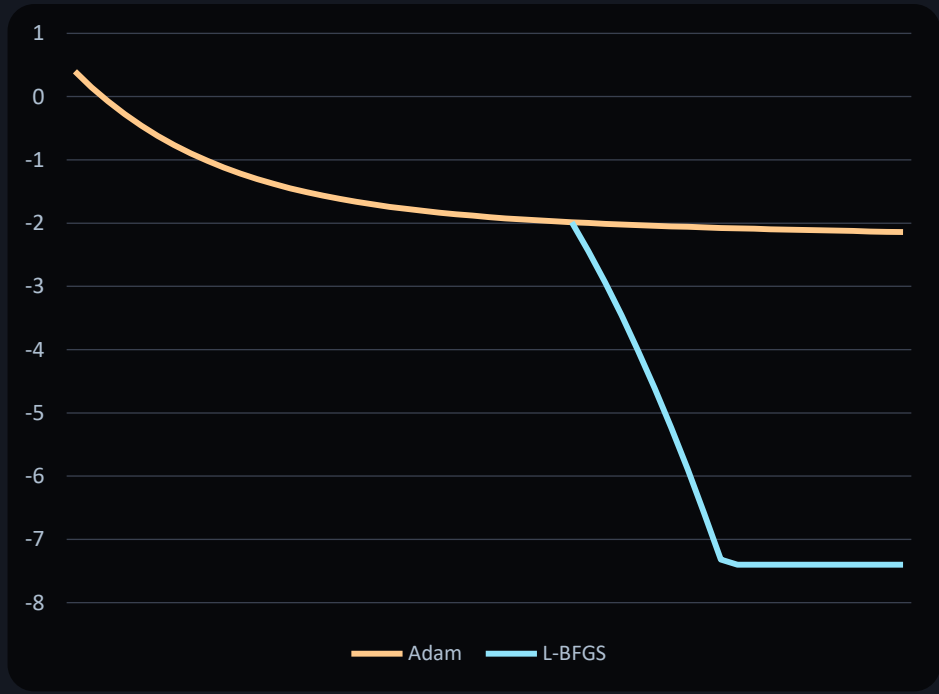
Two-Stage Optimisation: Adam then L-BFGS

- Adam is first-order and stochastic-friendly: robust to poor initialisation, good at escaping the flat regions of an untrained loss surface, but it stalls at moderate residual.
- L-BFGS is quasi-Newton: it builds curvature information from recent gradients and converges very fast near a minimum — but diverges if started far from one.
- Running Adam first and L-BFGS second exploits both. The switch is made on a fixed epoch count or on a loss plateau.
- Liu's schedule: Adam at lr = 1e-3 for ~201 epochs, then L-BFGS at lr = 1 with strong-Wolfe line search for ~11 epochs. An L-BFGS epoch costs 55–750× an Adam epoch — budget few, expect each to be slow.

MEASURED — SAME PROBLEM, NETWORK AND SEED

Two-stage	**6.5e-06**	18 s
Adam alone	**4.4e-04**	126 s

LOSS HISTORY (LOG SCALE)



switch at the plateau, not at a fixed loss value

Optimisers in Context: Why L-BFGS Works Here

- The two-stage recipe looks unusual against mainstream deep learning, where second-order methods are almost never used. The reason it works for PINNs is structural, not empirical.

SGD

the classical baseline

Momentum variants can find minima that generalise better (Wilson et al., 2017).
Slow on ill-conditioned losses.

Adam

adaptive moments

With tuned hyperparameters it matches SGD and converges faster (Choi et al., 2019).
Liu's stage 1, at lr = 1e-3.

L-BFGS

quasi-second-order

Rare in mainstream training — it needs a deterministic, full-batch loss.
A PINN supplies exactly that.

Why the PINN recipe differs: mainstream training is stochastic and minibatched, so curvature estimates are unreliable. A PINN's collocation set is fixed and its loss deterministic — which makes L-BFGS usable, and it is what delivers the final orders of magnitude.

Adaptive Sampling and Residual Normalisation

RESIDUAL-ADAPTIVE REFINEMENT

$$\mathbf{x}_{new} = \underset{\mathbf{x}\in S}{\operatorname{argmax}} |\mathcal{F}_{\hat{w}}(\mathbf{x})|$$

- Adaptive refinement evaluates the residual on a dense candidate set and adds points where it is largest
- so it reaches a target accuracy with far fewer points than uniform refinement

RESIDUAL NORMALISATION

$$\tilde{\mathcal{L}}_k = \frac{\mathcal{L}_k}{\mathcal{L}_k^{(0)} + \epsilon}$$

- normalisation rescales each loss term by its own magnitude, so none dominates on units alone
- both are stabilisers, not fixes

WHEN IT BECOMES NECESSARY, NOT MERELY HELPFUL

Burgers' equation at $\nu = 0.01/\pi$ develops a near-discontinuity by $t \approx 0.4$. Uniform sampling spends its points where the solution is flat.

BEFORE / AFTER REFINEMENT

dashed line: region of large $|\mathcal{F}\hat{w}|$

Liu Ch. 5.6, 5.8; Lu et al. (2021) DeepXDE

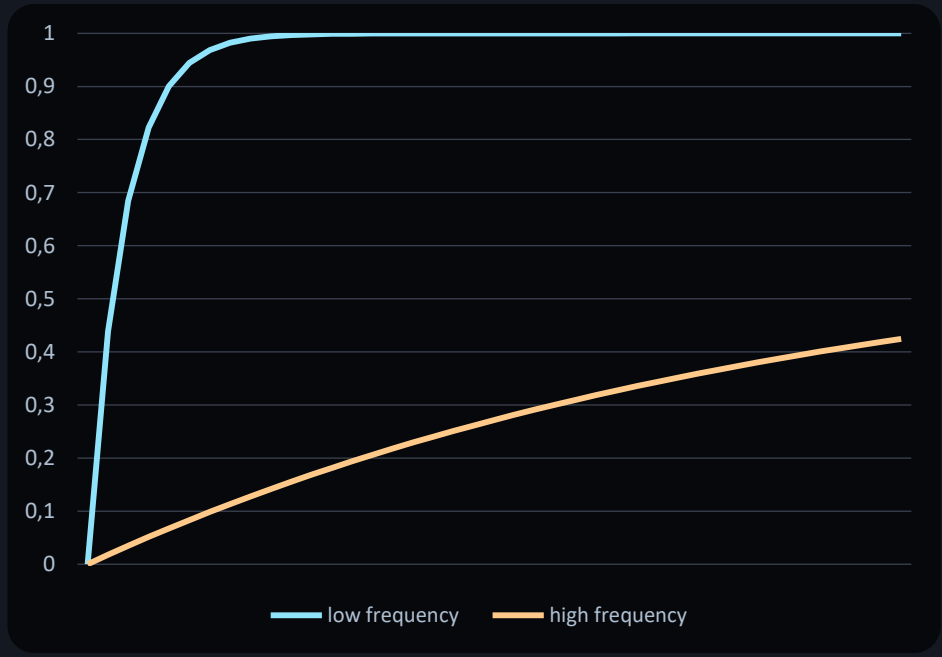
Spectral Bias: Low Frequencies Arrive First

TARGET USED IN THE DEMONSTRATION

$$u(x) = \sin(2\pi x) + \frac{1}{2}\sin(10\pi x)$$

- Gradient descent recovers the smooth part of a target long before its fine structure. In a 1-128-1 tanh network fitting the target above, the low-frequency mode reaches 90% of its amplitude by step 425; the high-frequency mode is still at 40% after 6 000 steps.
- This is the F-Principle. The neural tangent kernel explains it: descent converges fastest along high-eigenvalue directions, and those correspond to low frequencies.
- The consequence for a PINN is subtle and dangerous — the loss can be small and still falling while the fine structure of the solution is simply absent.
- Mitigations: Fourier feature embeddings, SIREN activations, multi-scale or hierarchical architectures.

AMPLITUDE RECOVERED, AS A FRACTION OF TARGET



Training step →. The gap between the two curves never closes on any budget a student will run.

Xu et al. (2019); Wang et al. (2022)

LIMITS

Failure Modes Worth Recognising

GRADIENT IMBALANCE

PDE-residual gradients overwhelm the BC and IC terms, so the network solves the equation while ignoring the conditions that make the answer unique.

Symptom: total loss falls, boundary error does not. Check per-term curves.

CAUSALITY VIOLATION

Nothing in the plain loss enforces time ordering, so training can fit late times before early ones have converged.

Fix: weight the time terms so the solution converges outward from $t = 0$.

STIFF AND MULTI-SCALE PDES

Widely separated scales produce an ill-conditioned optimisation landscape that traps both Adam and L-BFGS.

Symptom: oscillation near the minimum rather than convergence.

SENSITIVITY TO SETUP

Results shift with the collocation distribution and with weight initialisation — two runs of the same model need not agree.

Fix: fix the seed and report it, as Liu does throughout (seed 88).

Every item here is diagnosable from plots you already produce — per-term loss curves and a residual heatmap. Check the terms separately before changing the architecture.

Krishnapriyan et al. (2021); Wang et al. (2021)

SUMMARY

Key Takeaways

1 The ansatz	A PINN replaces the unknown field with a differentiable network. The Universal Approximation Theorem guarantees such a representation exists; it guarantees nothing about finding it.
2 The operator	Autograd supplies exact derivatives of that network, so the PDE, boundary and initial operators can all be evaluated pointwise without any discretisation.
3 The functional	Physics enters as a composite loss of squared residuals. $\mathcal{L}_p{}^d{}_e$ is always present; the remaining terms depend on the PDE class and on how much is hard-enforced.
4 The trade	Hard enforcement removes a loss term and the balancing problem with it — at the cost of a geometry-specific construction. This is the recurring design decision in L7.2.
5 The discipline	Capacity and sampling are coupled through $N_{sample} \geq P^*$, and optimisation is two-stage by default: Adam to explore, L-BFGS to converge.

Next — L7.2: the fundamental PDEs (Poisson, wave, Helmholtz, Burgers, transient heat) behind L8 heat, L9 fluids and batteries, L10 robotics and L11 power grids.

SUMMARY

PINNs at a Glance: Strengths, Limits, Best Fit

STRENGTH

Mesh-free and differentiable end to end. Fuses sparse data with physics. Returns a continuous solution evaluable at any point, with no interpolation.

WEAKNESS

Spectral bias and gradient pathologies. Training cost far above a comparable classical solve for a single forward problem.

BEST FIT

Sparse-data regimes, inverse and parameter-identification problems, smooth to moderately oscillatory PDEs, repeated parametric studies.

CAUTION

Stiff, high-frequency and multi-scale problems need special treatment — or a classical solver. Judge the method against the problem, not the fashion.

Physics enters through the loss, not the architecture — which is why the same network, unchanged, solves heat conduction in L8 and a battery thermal model in a project.

Liu Ch. 2.10; Raissi et al. (2019)

OVER TO YOU

Questions

TEN TO TEST WHETHER TODAY LANDED

- **what does the PDE residual measure** — and where is it evaluated?
- why must the activation be smooth, stated from the order of the equation?
- what does universal approximation not promise?
- **soft or hard enforcement** — what does each cost you?
- why do the collocation points carry no boundary information?
- what is the weight between two loss terms, physically?

THE ONE WITH NO SETTLED ANSWER

There is no validation set here. How will you know when to stop?

BEFORE L7.2

Ex_07.1 notebooks 01 and 02. Bring both error figures.

EXERCISE

Ex_07 — PINN for the 2-D Poisson Problem

STUDENT TASKS

1. Implement the PINN; generate interior and boundary collocation points.
2. Train a soft-BC model; record the final PDE and BC losses separately.
3. Train a hard-BC model using the trial solution.
4. Compare both against the analytical solution on a regular grid.
5. Report relative L_2 error and maximum absolute error.

QUESTIONS — AND WHERE THIS DECK ANSWERS THEM

Why is a smooth activation preferred?	slide 11
What is the role of automatic differentiation?	slide 14
What does hard enforcement gain over a weighted BC loss?	slides 22, 23
PDE loss small but boundary inaccurate — what to check?	slides 26, 36
How do collocation count and spread affect the solution?	slides 9, 28

Measured reference results for both variants are on slide 23 — attempt the exercise before reading them.

APPENDIX

Source Map and Merge Log

- This deck merges the mathematical spine of the L7.1 rewrite with the concrete measurements, course framing and exercise material from the L7_PINN deck.

L7.1 rewrite (28 slides)	spine — five-stage flow, typeset equations, diagrams
L7_PINN (43 slides)	merged in — measured benchmarks, course arc, exercise, limits
Liu (2025), Ch. 2–6	primary source for all formulae and logged numbers
Raissi et al. (2019)	operator form, inverse problem, Burgers benchmark

ADDED IN THIS MERGE

Course arc L8–L12 · modelling spectrum · measured hard-vs-soft benchmark · Liu's real network configurations · Adam/L-BFGS logged numbers · optimiser context · spectral bias · failure modes · strengths and limits · Ex_07 exercise

SOURCE CORRECTIONS CARRIED FORWARD

P* is the pseudo-dimension $(p+1) + (N_n+1)N_l$, not the trainable-parameter count — both source decks are corrected here.

Chapter pointers: the samples–neurons theorem is Liu Ch. 2.5.2 Eq. (2.19), not Ch. 2.3; train_adam_lbfgs is Ch. 2.7.5–2.7.6, not Ch. 2.4.